Backend Briefing - Fokus Lead Backend (TraceIt / TraceIt)

Dokumen ini disusun khusus untuk persiapan kamu sebagai Lead Backend, berdasarkan source code dan dokumen yang ada di repository saat ini.

1. Gambaran Arsitektur Backend Saat Ini

Sistem backend berjalan dengan FastAPI + SQLAlchemy + PostgreSQL + JWT auth.

Alur umum:

1. Request masuk ke endpoint FastAPI di main.py.
2. Validasi body/query dilakukan oleh schema Pydantic (schemas.py).
3. Business logic dan query database dijalankan di crud.py.
4. Koneksi DB dan lifecycle session diatur oleh database.py.
5. Proteksi endpoint (admin/pengguna) dilakukan oleh dependency auth.py.
6. Data dipetakan ke tabel melalui model ORM di models.py.

2. Logika dan Fungsi Setiap File Backend

backend/main.py

Fungsi utama:

1. Entry point API (FastAPI app).
2. Registrasi seluruh endpoint.
3. Konfigurasi CORS dari environment variable ALLOWED_ORIGINS.
4. Startup maintenance ringan untuk kompatibilitas schema lama.

Logika penting:

1. Base.metadata.create_all(bind=engine) membuat tabel jika belum ada.
2. Saat startup, ada 3 maintenance:
    - ensure_schema_compatibility() dari models.py untuk menambah kolom id_pengguna di riwayat_donor jika belum ada.
    - _ensure_backend_schema_compatibility() untuk cek/repair kolom pendonor.email dan tipe no_telepon.
    - _cleanup_duplicate_admins() agar admin hanya satu akun.
3. Endpoint auth dibuat dua jalur untuk sebagian route:
    - tanpa /api, contoh /auth/admin/login
    - dengan /api, contoh /api/auth/admin/login Ini dibuat untuk kompatibilitas alur frontend/proxy.

backend/database.py

Fungsi utama:

1. Menentukan env file yang dipakai (.env atau .env.docker).

2. Membuat SQLAlchemy engine + SessionLocal.
3. Menyediakan dependency get_db() untuk buka/tutup session per request.

Logika penting:

1. Prioritas env file:
    - ENV_FILE (kalau di-set)
    - .env.docker (kalau berjalan di container dan file ada)
    - default .env
2. load_dotenv(..., override=False) tidak menimpa env yang sudah diinject Docker/CI.
3. DATABASE_URL wajib ada, kalau tidak backend langsung raise error.

backend/models.py

Fungsi utama:

1. Definisi tabel ORM: Admin, Pengguna, Pendonor, RiwayatDonor.
2. Definisi enum domain (golongan darah, jenis kelamin).
3. Relasi antar tabel dan cascade rule.

Logika penting:

1. RiwayatDonor punya id_pengguna nullable untuk melacak kepemilikan input pengguna.
2. Relasi Pendonor -> RiwayatDonor pakai cascade all, delete-orphan (hapus pendonor ikut hapus riwayat terkait).
3. ensure_schema_compatibility() menambah id_pengguna di schema lama agar tidak perlu migrasi manual langsung.

backend/schemas.py

Fungsi utama:

1. Kontrak request/response API.
2. Validasi field dan batasan data.
3. Menjaga format output konsisten untuk frontend.

Logika penting:

1. Validasi password di AdminCreate/PenggunaCreate:
    - wajib huruf besar
    - wajib huruf kecil
    - wajib angka
2. Validasi data pendonor ketat (umur, berat, tinggi, no telepon, dll).
3. Response model list memakai format standar:
    - PendonorListResponse: { total, pendonor }
    - RiwayatDonorListResponse: { total, riwayat_donor }

backend/auth.py

Fungsi utama:

1. Hash dan verifikasi password (bcrypt via passlib).
2. Buat dan decode JWT.
3. Dependency otorisasi role-based:
    - get_current_admin
    - get_current_pengguna

Logika penting:

1. claim sub dipaksa string saat create token untuk mencegah gagal decode.
2. Token harus punya user_type sesuai endpoint (admin/pengguna).
3. Jika token invalid/expired atau role salah, backend kirim 401.

backend/crud.py

Fungsi utama:

1. Seluruh business logic database.
2. Query CRUD untuk admin, pengguna, pendonor, riwayat donor.
3. Sinkronisasi data antar entitas berdasarkan email.

Logika penting (yang sering ditanya penguji):

1. create_admin() membatasi hanya satu admin sistem.
2. create_pengguna() melakukan linking otomatis:
    - jika ada pendonor dengan email sama, maka dibuat riwayat donor pending milik pengguna.
3. create_pendonor() juga kebalikannya:
    - jika email pendonor sudah punya akun pengguna, otomatis buat riwayat donor pending.
4. update_pendonor() menjaga linking tetap sinkron jika email berubah.
5. verifikasi_riwayat_donor():
    - jika status diverifikasi true, total_donor pendonor dinaikkan +1.
6. get_public_blood_stock() hanya hitung stok dari riwayat donor yang sudah terverifikasi.

backend/Dockerfile

Fungsi utama:

1. Build image backend multi-stage (builder + runtime) berbasis python:3.12-alpine.
2. Menjalankan wait-for-db.sh sebelum uvicorn start.
3. Menjalankan app sebagai non-root user.
4. Menyediakan healthcheck /health.

Logika penting:

1. Stage builder: install dependency ke virtualenv /opt/venv.
2. Stage production: copy venv + source code, install curl dan postgresql-client.
3. ENTRY command menunggu DB siap dulu, baru uvicorn jalan.

backend/scripts/wait-for-db.sh

Fungsi utama:

1. Menahan startup backend sampai PostgreSQL siap.
2. Parse host/port/user/db dari DATABASE_URL jika DB_* tidak diset.
3. Retry dengan timeout terukur.

Ini penting untuk menghindari race condition saat container backend naik lebih cepat dari DB.

3. Alur Backend ke Frontend dan Docker

A. Alur ke Frontend (Development)

Bukti dari frontend/src/services/api.js + vite.config.js:

1. Frontend pakai base URL /api.
2. Vite proxy meneruskan /api ke http://localhost:8000.
3. Rewrite rule:
    - /api/public/* tidak dihapus prefix /api (tetap /api/public/...)
    - route lain dihapus prefix /api (misalnya /api/pendonor -> /pendonor)

Dampak ke backend:

1. Endpoint /api/public/blood-stock harus memang ada (dan ada di main.py).
2. Endpoint lain cukup ada versi tanpa /api (misal /pendonor, /pengguna/me, /riwayat-donor).
3. Untuk auth admin/pengguna, backend sudah menyediakan dua jalur (dengan dan tanpa /api), jadi aman untuk kompatibilitas.

B. Alur Docker Compose

Bukti dari docker-compose.yml:

1. Service db: postgres:15, nama container tracelt-db.
2. Service backend: image avhillacatton/tracelt-backend:v2-be, env DATABASE_URL menunjuk ke db:5432.
3. Service frontend: image avhillacatton/tracelt-frontend:v1-fe.

Catatan penting akurasi:

1. Di compose ini backend memakai image registry, bukan build lokal dari folder backend.
2. Frontend juga memakai image registry.
3. Jadi perilaku runtime actual saat compose up sangat bergantung pada isi image di registry, tidak selalu identik 100% dengan source terbaru di repo.

4. Endpoint Backend Saat Ini (Dari Kode Aktif)

Daftar endpoint aktif di backend/main.py:

Health & Info

1. GET /health
2. GET /info

Public

1. GET /api/public/blood-stock

Auth Admin

1. POST /auth/admin/register
2. POST /api/auth/admin/register
3. POST /auth/admin/login
4. POST /api/auth/admin/login

Auth Pengguna

1. POST /auth/pengguna/register
2. POST /api/auth/pengguna/register
3. POST /auth/pengguna/login
4. POST /api/auth/pengguna/login

Pengguna Profile & Riwayat (token pengguna)

1. GET /pengguna/me
2. POST /pengguna/riwayat-donor
3. GET /pengguna/riwayat-donor
4. GET /pengguna/riwayat-donor/{riwayat_id}
5. PUT /pengguna/riwayat-donor/{riwayat_id}
6. DELETE /pengguna/riwayat-donor/{riwayat_id}

Pendonor

1. POST /pendonor
2. GET /pendonor
3. GET /pendonor/{pendonor_id}
4. PUT /pendonor/{pendonor_id}
5. DELETE /pendonor/{pendonor_id} (butuh admin token)

Riwayat Donor (global/admin workflow)

1. POST /riwayat-donor
2. GET /riwayat-donor
3. GET /riwayat-donor/pendonor/{pendonor_id}
4. GET /riwayat-donor/{riwayat_id}
5. POST /riwayat-donor/{riwayat_id}/verifikasi (butuh admin token)

5. Endpoints Berhasil Semua atau Tidak?

Penilaian akurat berdasarkan data yang tersedia:

Yang punya bukti lulus (documented)

1. docs/api-test-results.md menyatakan endpoint /items era lama lulus (CRUD, pagination, stats).
2. README.md menyatakan pengujian docker service berhasil naik.

Yang tidak bisa dipastikan lulus 100% dari bukti sekarang

1. Endpoint aktif saat ini sudah bergeser ke domain pendonor/pengguna/riwayat, bukan lagi /items.
2. Test script backend/test_pagination.py masih menguji /items, jadi tidak merepresentasikan endpoint terbaru.
3. Tidak ada laporan uji formal terpisah di docs untuk seluruh endpoint terbaru auth + pendonor + riwayat.

Kesimpulan objektif:

1. Secara kode, endpoint terbaru terlihat lengkap dan konsisten.
2. Secara bukti pengujian tertulis, coverage endpoint terbaru belum penuh.
3. Jadi jawaban yang paling akurat: belum ada bukti dokumenter bahwa semua endpoint terbaru sudah teruji penuh, meskipun implementasinya ada.

6. Environment Variables yang Digunakan

Backend

1. DATABASE_URL
    - Koneksi PostgreSQL.
2. ENV_FILE (opsional)
    - Menentukan file env spesifik.
3. SECRET_KEY
    - Secret signing JWT.
4. ALGORITHM
    - Algoritma JWT (contoh HS256).
5. ACCESS_TOKEN_EXPIRE_MINUTES
    - Durasi token.
6. ALLOWED_ORIGINS
    - Whitelist CORS.
7. WAIT_TIMEOUT_SECONDS (opsional, startup script)
8. WAIT_INTERVAL_SECONDS (opsional, startup script)
9. DB_HOST, DB_PORT, DB_USER, DB_NAME (opsional, startup script)

Frontend

1. VITE_API_URL (terdapat di .env.example frontend)
2. GEMINI_API_KEY (dipakai pada define di vite config)

Catatan:

- Nilai rahasia (secret/password) sebaiknya tidak dipresentasikan ke penguji, cukup sebut nama variabel dan fungsinya.

7. Versi Technology Stack (Berdasarkan Repository)

Backend Runtime/Library

1. FastAPI 0.115.0

2. Uvicorn 0.30.0
3. SQLAlchemy 2.0.35
4. psycopg2-binary 2.9.10
5. python-dotenv 1.0.1
6. pydantic[email] 2.9.0
7. python-jose[cryptography] 3.3.0
8. passlib[bcrypt] 1.7.4
9. bcrypt 4.0.1

Container / Infra

1. Backend base image: python:3.12-alpine (multi-stage)
2. Frontend build image: node:20-slim
3. Frontend runtime image: nginx:alpine
4. Docker Compose DB image: postgres:15

Frontend Core

1. React 19.0.0
2. Vite 6.2.0
3. Axios 1.14.0
4. React Router DOM 7.14.0

8. Daftar Pertanyaan Penguji yang Sangat Mungkin Ditanyakan (Lead Backend)

A. Dasar Arsitektur dan Alur

1. Jelaskan alur request dari frontend sampai data tersimpan di PostgreSQL.
2. Kenapa business logic dipisah ke crud.py dan tidak ditulis semua di main.py?
3. Kenapa pakai ORM, bukan raw SQL untuk semua operasi?

B. Validasi dan Data Modeling

1. Bedanya model SQLAlchemy dan schema Pydantic apa?
2. Validasi apa saja yang kamu terapkan untuk password dan data pendonor?
3. Kenapa kolom id_pengguna di riwayat_donor dibuat nullable?

C. Authentication dan Authorization

1. Bedanya autentikasi dan otorisasi di sistem kamu apa?
2. Kenapa claim sub di JWT kamu ubah jadi string?
3. Bagaimana sistem memastikan endpoint tertentu hanya untuk admin?
4. Apa risiko kalau SECRET_KEY bocor?

D. Database dan Konsistensi Data

1. Kenapa ada linking otomatis pendonor <-> pengguna berdasarkan email?
2. Apa dampak bisnis dari verifikasi riwayat donor terhadap total_donor?

3. Jelaskan fungsi startup maintenance schema compatibility.
4. Apa risiko pakai create_all tanpa migration tool formal?

E. Docker dan Cloud Readiness

1. Kenapa Dockerfile backend memakai multi-stage build?
2. Kenapa backend perlu wait-for-db.sh padahal ada depends_on di compose?
3. Apa fungsi healthcheck backend?
4. Kenapa service di compose diarahkan ke container name/hostname, bukan localhost?

F. API Contract dan Integrasi Frontend

1. Kenapa endpoint auth disediakan dua versi (dengan dan tanpa /api)?
2. Jelaskan bagaimana Vite proxy rewrite bekerja untuk route /api/public/* dan route lain.
3. Apa yang terjadi kalau frontend memanggil endpoint yang response modelnya tidak konsisten?

G. Pengujian dan Reliability

1. Bukti apa yang kamu punya bahwa endpoint berjalan?
2. Bagaimana kamu membuktikan endpoint terbaru sudah benar-benar teruji?
3. Jika ada bug 401 random pada pengguna, langkah debug kamu apa saja?

H. Cloud Computing Concept

1. Bagaimana sistem ini mencerminkan karakteristik cloud computing (on-demand, scalable, measured)?
2. Kalau dipindah ke PaaS, variabel mana yang wajib dikonfigurasi dulu?
3. Strategi scaling backend ini bagaimana jika trafik naik 10x?

8B. Jawaban Singkat untuk Tiap Pertanyaan Penguji

A. Dasar Arsitektur dan Alur - Jawaban

1. Frontend kirim HTTP ke FastAPI, divalidasi schema, diproses di crud, disimpan lewat SQLAlchemy ke PostgreSQL, lalu response JSON dikirim balik.
2. Supaya rapi dan mudah maintain: main.py fokus routing, crud.py fokus business logic, jadi testing dan debugging lebih gampang.
3. ORM dipakai untuk produktivitas, keamanan query, dan konsistensi model; raw SQL hanya dipakai bila perlu optimasi khusus.

B. Validasi dan Data Modeling - Jawaban

1. Model SQLAlchemy adalah representasi tabel database, schema Pydantic adalah kontrak validasi request/response API.
2. Password wajib ada huruf besar, kecil, angka; data pendonor divalidasi umur, berat/tinggi, no telepon, email, dan enum golongan darah.
3. Karena tidak semua riwayat donor pasti berasal dari akun pengguna login; ada data yang bisa berasal dari input admin/public flow.

C. Authentication dan Authorization - Jawaban

1. Autentikasi = verifikasi identitas (login), otorisasi = cek hak akses endpoint berdasarkan role.
2. Karena claim sub di JWT standar diperlakukan string; ini mencegah error parse saat decode.
3. Endpoint memakai dependency get_current_admin/get_current_pengguna yang cek token + user_type.
4. Jika SECRET_KEY bocor, token bisa dipalsukan; mitigasi: rotate key, invalidate token lama, ganti secret via env.

D. Database dan Konsistensi Data - Jawaban

1. Agar data pendonor dan akun pengguna yang email-nya sama otomatis tersinkron, tidak duplikasi manual.
2. Saat riwayat diverifikasi true, total_donor naik 1; ini memengaruhi statistik dan status donor aktif.
3. Startup compatibility dipakai untuk penyesuaian schema ringan tanpa migrasi penuh agar kode baru tetap jalan di DB lama.
4. Risiko create_all tanpa migration: perubahan schema kompleks sulit terkontrol, rawan drift antar environment.

E. Docker dan Cloud Readiness - Jawaban

1. Multi-stage bikin image lebih kecil dan aman karena dependency build tidak ikut ke final image.
2. depends_on hanya urutan start, bukan readiness; wait-for-db memastikan DB benar-benar siap menerima koneksi.
3. Healthcheck memastikan service backend benar-benar sehat (endpoint /health bisa diakses).
4. Karena komunikasi antar container harus pakai hostname service Docker network, bukan localhost host machine.

F. API Contract dan Integrasi Frontend - Jawaban

1. Untuk kompatibilitas: ada client yang akses /auth/... langsung, ada yang melalui /api/... dari proxy.
2. Vite mempertahankan /api/public/* apa adanya, sementara route /api lain di-strip menjadi route backend tanpa prefix /api.
3. Frontend bisa error parsing/render, status handling kacau, dan bug runtime; karena itu response model harus konsisten.

G. Pengujian dan Reliability - Jawaban

1. Bukti saat ini: dokumen uji endpoint lama (/items), dokumen docker up, serta implementasi endpoint terbaru di kode.
2. Buat test matrix endpoint terbaru (auth, pendonor, riwayat), jalankan via Swagger/Postman/pytest, lalu dokumentasikan hasil status code + payload.
3. Debug 401: cek header Bearer, cek token expiry, cek SECRET_KEY/ALGORITHM, cek claim user_type, cek dependency endpoint.

H. Cloud Computing Concept - Jawaban

1. On-demand: service bisa naik lewat compose, scalable: bisa scale service backend, measured: penggunaan resource bisa dipantau di container metrics/logs.
2. Prioritas env di PaaS: DATABASE_URL, SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES, ALLOWED_ORIGINS.

3. Strategi 10x trafik: scale backend horizontal, pakai reverse proxy/load balancer, optimasi DB index/query, tambah caching dan observability.

9. Jawaban Cepat yang Harus Kamu Hafal (Cheat Answers)

1. Kenapa split file backend?
    - Untuk separation of concerns: router di main.py, business logic di crud.py, model di models.py, validasi di schemas.py, auth di auth.py, koneksi di database.py.
2. Kenapa pakai JWT?
    - Stateless, cocok untuk API dan scaling horizontal, mudah dipakai antar service.
3. Kenapa ada wait-for-db?
    - Karena service start order tidak menjamin DB benar-benar siap menerima koneksi.
4. Bedanya admin dan pengguna?
    - Role dibedakan lewat claim user_type pada JWT dan dependency terpisah get_current_admin/get_current_pengguna.
5. Endpoint semua sukses?
    - Berdasarkan bukti dokumen, endpoint lama /items sudah teruji. Endpoint domain terbaru implementasinya lengkap di kode, tetapi belum ada satu dokumen uji komprehensif yang membuktikan 100% seluruh endpoint terbaru.

10. Rekomendasi Persiapan Viva 15 Menit

1. Buka docs dan jelaskan arsitektur dulu (2 menit).
2. Tunjukkan endpoint auth + pendonor + riwayat di Swagger (5 menit).
3. Demo satu alur end-to-end:
    - register pengguna
    - login pengguna
    - create riwayat pengguna
    - verifikasi oleh admin
    - cek perubahan status/total donor (5 menit)
4. Tutup dengan penjelasan docker readiness (healthcheck + wait-for-db + env vars) (3 menit).

11. Paket Demo Besok: Alur Auth dan JWT (Authentication + Authorization)

Bagian ini dibuat biar kamu bisa demo code dengan runtut, singkat, dan meyakinkan.

A. Narasi Pembuka (30-45 detik)

Kalimat pembuka yang aman dipakai:

1. Authentication di sistem ini adalah proses login admin/pengguna untuk verifikasi identitas.
2. Setelah login sukses, backend mengeluarkan JWT access token.

3. Authorization dilakukan dengan memeriksa isi token (khususnya user_type) lewat dependency FastAPI.
4. Jadi, siapa user-nya valid (authentication), lalu dicek boleh akses endpoint mana (authorization).

B. Urutan Demo Endpoint (Live di Swagger/Postman)

Urutan ideal supaya penguji langsung paham:

1. POST /auth/pengguna/register
2. POST /auth/pengguna/login
3. Ambil access_token dari response login
4. Akses endpoint pengguna yang butuh token:
    - GET /pengguna/me
    - GET /pengguna/riwayat-donor
5. Tunjukkan endpoint admin-only:
    - POST /auth/admin/login
    - POST /riwayat-donor/{riwayat_id}/verifikasi (butuh token admin)
6. Uji authorization gagal:
    - pakai token pengguna ke endpoint admin-only => harus 401

C. Titik Kode yang Kamu Tunjukkan Saat Demo

1) Pembuatan token JWT (Authentication result)

Lokasi: backend/auth.py

Hal yang dijelaskan:

1. create_access_token() menerima data user (sub + user_type).
2. sub dipaksa string agar aman saat decode.
3. exp ditambahkan dari ACCESS_TOKEN_EXPIRE_MINUTES.

Contoh potongan logika:

```
def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) ->
str:
   to_encode = data.copy()
   if "sub" in to_encode:
      to_encode["sub"] = str(to_encode["sub"])
   expire = datetime.now(timezone.utc) + (expires_delta or
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
   to_encode.update({"exp": expire})
   return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

2) Login endpoint yang mengeluarkan token

Lokasi: backend/main.py

Hal yang dijelaskan:

1. Endpoint login hanya verifikasi email + password.
2. Jika valid, generate token berisi sub dan user_type.
3. Response kembali ke frontend dalam format bearer token.

Contoh admin:

```
@app.post("/auth/admin/login", response_model=TokenResponse)
def login_admin(login_data: LoginRequest, db: Session = Depends(get_db)):
    admin = crud.authenticate_admin(db=db, email=login_data.email,
password=login_data.password)
    if not admin:
        raise HTTPException(status_code=401, detail="Email atau password admin
salah")

    token = create_access_token(data={"sub": admin.id_admin, "user_type": "admin"})
    return {
        "access_token": token,
        "token_type": "bearer",
        "user_type": "admin",
    }
```

3) Validasi token (Authentication check)

Lokasi: backend/auth.py

Hal yang dijelaskan:

1. decode_token() memverifikasi signature token dengan SECRET_KEY.
2. Jika token invalid/expired, backend kirim 401.

```
def decode_token(token: str) -> dict:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token tidak valid atau sudah expired",
            headers={"WWW-Authenticate": "Bearer"},
        )
```

4) Role check (Authorization)

Lokasi: backend/auth.py

Hal yang dijelaskan:

1. get_current_admin() cek user_type harus admin.
2. get_current_pengguna() cek user_type harus pengguna.

3. Kalau role salah, langsung 401 walaupun token valid.

Contoh inti admin-only:

```
def get_current_admin(token: str = Depends(oauth2_scheme), db: Session =
Depends(get_db)) -> Admin:
   payload = decode_token(token)
   admin_id = _parse_subject_id(payload.get("sub"))
   user_type: str = payload.get("user_type")

   if user_type != "admin" or admin_id is None:
      raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Hanya
admin yang bisa akses endpoint ini")

   admin = db.query(Admin).filter(Admin.id_admin == admin_id).first()
   if admin is None:
      raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Admin
tidak ditemukan")
   return admin
```

5) Endpoint yang memakai authorization dependency

Lokasi: backend/main.py

Contoh endpoint admin-only:

```
@app.post("/riwayat-donor/{riwayat_id}/verifikasi",
response_model=RiwayatDonorResponse)
def verifikasi_riwayat_donor(
   riwayat_id: int,
   verifikasi_data: RiwayatDonorVerifikasi,
   db: Session = Depends(get_db),
   current_admin: Admin = Depends(get_current_admin),
):
   ...
```

Point bicara:

1. current_admin hanya akan terisi jika token valid + role admin.
2. Ini adalah inti authorization enforcement di endpoint.

D. Hubungan Frontend dengan JWT

Lokasi: frontend/src/services/api.js

Yang kamu jelaskan:

1. Frontend menyimpan token di localStorage:
   - admin_token

    - user_token
2. Axios interceptor otomatis menyisipkan header Authorization: Bearer .
3. Token dipilih berdasarkan prefix endpoint (pengguna vs admin).

Contoh inti interceptor:

```
api.interceptors.request.use((config) => {
  const adminToken = localStorage.getItem('admin_token');
  const userToken = localStorage.getItem('user_token');
  ...
  if (token) {
   config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

E. Skenario Demo Error (Wajib untuk menunjukkan paham)

Satu skenario kuat:

1. Login sebagai pengguna (dapat user token).
2. Panggil endpoint admin-only verifikasi donor.
3. Tunjukkan hasil 401.
4. Jelaskan: authentication lolos (token valid), tetapi authorization gagal (role bukan admin).

F. Kalimat Penutup Demo (15 detik)

"Jadi implementasi JWT di TraceIt memisahkan dua lapisan keamanan: pertama verifikasi identitas lewat login dan token, kedua kontrol hak akses lewat role-based dependency pada setiap endpoint penting."

12. Mapping Materi Modul Dosen -> Implementasi di TraceIt (Untuk Jawaban Ujian)

Bagian ini menyambungkan teori modul dosen dengan praktik di codebase kamu, supaya jawaban terdengar akademis sekaligus praktis.

Minggu 1-2: Cloud Computing & REST API

A. Definisi Cloud Computing + 5 Karakteristik NIST

Definisi singkat: Cloud computing adalah model penyediaan resource komputasi (compute, storage, network, software) melalui internet secara elastis dan terukur.

5 karakteristik NIST dan konteks TraceIt:

1. On-demand self-service
    - Service bisa di-up cepat lewat Docker Compose tanpa setup manual panjang.
2. Broad network access
    - API diakses lewat HTTP/HTTPS oleh frontend (browser) dan tools API testing.

3. Resource pooling
    - Container backend, frontend, dan database berbagi resource host secara terisolasi.
4. Rapid elasticity
    - Backend bisa di-scale horizontal (beberapa instance FastAPI di belakang reverse proxy/load balancer).
5. Measured service
    - Penggunaan CPU, memory, dan network dapat dimonitor dari metrics/log container.

Jawaban cepat saat ditanya: "TraceIt sudah mengikuti pola cloud-native dasar: service ter-containerisasi, konfigurasi lewat environment variable, dan siap di-scale serta dipantau resource-nya."

B. Model Layanan: IaaS, PaaS, SaaS (Perbedaan dan Contoh)

1. IaaS
    - User mengelola VM, OS, runtime, aplikasi.
    - Contoh: AWS EC2, GCP Compute Engine.
2. PaaS
    - Provider mengelola infra + runtime, user fokus deploy aplikasi.
    - Contoh: Render, Railway, Heroku.
3. SaaS
    - User langsung pakai aplikasi jadi melalui web.
    - Contoh: Gmail, Google Docs.

Posisi TraceIt:

1. Saat ini arsitektur dan Docker setup cocok untuk deployment ke IaaS maupun PaaS.
2. Jika dipublikasikan sebagai aplikasi untuk end-user, produknya akan menjadi SaaS domain kesehatan donor darah.

C. Cloud-Native vs Traditional

Perbandingan singkat:

1. Traditional
    - Monolitik, setup server manual, scaling sering vertikal.
2. Cloud-native
    - Containerized, env-based config, otomatisasi deployment, scaling horizontal.

Bukti cloud-native pada TraceIt:

1. Backend dan frontend pakai Docker image.
2. Konfigurasi sensitif pakai env vars (DATABASE_URL, SECRET_KEY, ALLOWED_ORIGINS, dll).
3. Ada healthcheck endpoint dan wait-for-db untuk startup reliability.

D. HTTP Methods dan Mapping CRUD

Mapping teori -> praktik:

1. CREATE -> POST
    - POST /pendonor

    - POST /riwayat-donor
2. READ -> GET
    - GET /pendonor
    - GET /riwayat-donor
    - GET /pengguna/me
3. UPDATE -> PUT
    - PUT /pendonor/{pendonor_id}
    - PUT /pengguna/riwayat-donor/{riwayat_id}
4. DELETE -> DELETE
    - DELETE /pendonor/{pendonor_id}
    - DELETE /pengguna/riwayat-donor/{riwayat_id}

Jawaban cepat: "Saya konsisten pakai HTTP verb sesuai semantik CRUD, jadi kontrak API jelas dan mudah dipahami frontend maupun penguji."

E. HTTP Status Codes (200, 201, 204, 400, 404, 422)

Contoh yang relevan di sistem:

1. 200 OK
    - Request GET berhasil, data dikembalikan normal.
2. 201 Created
    - Berhasil membuat resource baru (register/login tertentu, create data).
3. 204 No Content
    - Berhasil delete tanpa body response.
4. 400 Bad Request
    - Rule bisnis tidak terpenuhi (misalnya data yang sudah diverifikasi tidak boleh diubah).
5. 404 Not Found
    - Resource dengan ID tertentu tidak ditemukan.
6. 422 Unprocessable Entity
    - Validasi Pydantic gagal (format/body request tidak sesuai schema).

F. REST API Principles

Prinsip utama dan penerapan:

1. Resource-oriented URL
    - Endpoint berbasis resource: pendonor, riwayat-donor, pengguna.
2. Stateless
    - Server tidak simpan session login state; autentikasi berbasis JWT di setiap request.
3. Uniform interface
    - Metode HTTP dan format JSON konsisten.
4. Layered system
    - Frontend -> API -> DB dipisah per layer, dipertegas oleh Docker service.

Catatan jujur saat ujian: "Sebagian endpoint auth masih disediakan dua jalur (/auth dan /api/auth) untuk kompatibilitas client, tapi prinsip REST utama tetap dijaga."

Minggu 4: Full-Stack Integration

A. CORS: Apa, Kenapa, dan Konfigurasi

Konsep:

1. CORS adalah mekanisme browser untuk mengontrol cross-origin request.
2. Tanpa CORS yang benar, frontend bisa kena blokir walaupun API hidup.

Penerapan di TraceIt:

1. FastAPI menambahkan CORSMiddleware.
2. Origin dibaca dari env ALLOWED_ORIGINS.
3. allow_methods dan allow_headers dibuka agar integrasi frontend tidak terhambat saat development.

Jawaban cepat: "CORS saya jadikan configurable via env supaya aman dan fleksibel antara local, staging, dan production."

B. JWT: Struktur Token dan Alur Authentication

Struktur JWT:

1. Header (alg, typ)
2. Payload (sub, user_type, exp)
3. Signature (ditandatangani SECRET_KEY)

Alur di sistem:

1. User login -> backend verifikasi credential.
2. Backend generate access token JWT.
3. Frontend menyimpan token dan kirim di header Authorization: Bearer.
4. Dependency backend decode token dan cek role sebelum endpoint dijalankan.

Perbedaan authn vs authz di jawaban:

1. Authentication: memastikan siapa user-nya (login + token valid).
2. Authorization: memastikan user boleh akses endpoint tertentu (cek user_type admin/pengguna).

C. Environment Variables dan 12-Factor App

Kenapa penting:

1. Rahasia tidak di-hardcode.
2. Konfigurasi antar environment (dev/staging/prod) bisa berbeda tanpa ubah kode.

Implementasi di TraceIt:

1. DATABASE_URL untuk koneksi DB.
2. SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES untuk JWT.
3. ALLOWED_ORIGINS untuk CORS policy.
4. Pemilihan env file di database.py menunjukkan pemisahan config dari code.

Relasi ke 12-Factor:

1. Config in environment: sudah diterapkan.
2. Dev/prod parity: dibantu Docker image dan compose.
3. Disposability/readiness: dibantu healthcheck dan wait-for-db.

D. Keamanan: Password Hashing dan Token Expiry

Password hashing:

1. Password tidak disimpan plaintext.
2. Menggunakan bcrypt via passlib untuk hash + verifikasi.

Token expiry:

1. Token JWT punya claim exp.
2. Token kadaluarsa akan ditolak saat decode (401).
3. Ini membatasi risiko jika token bocor.

Jawaban cepat: "Keamanan minimum yang wajib saya jaga adalah hash password, secret key yang aman, validasi role, dan masa berlaku token yang terbatas."

13. Bank Jawaban Siap Ucap (Sesuai Modul)

Gunakan jawaban ini saat dosen tanya cepat.

1. Apa itu cloud computing pada konteks project kamu?
    - Cloud computing di project ini berarti service backend, frontend, dan database dikelola sebagai layanan terpisah yang bisa dideploy elastis dan dikonfigurasi lewat environment.
2. Kenapa endpoint kamu RESTful?
    - Karena resource dipisah jelas (pendonor, riwayat), method HTTP sesuai CRUD, response JSON konsisten, dan endpoint bersifat stateless dengan JWT.
3. Kenapa butuh CORS?
    - Karena frontend dan backend berjalan di origin berbeda saat development; CORS mengizinkan origin tertentu agar browser tidak memblokir request.
4. Bedanya 401 dan 403 di sistem kamu?
    - 401 dipakai saat token tidak valid/expired atau role tidak sesuai dependency. 403 biasanya untuk user yang valid tapi dilarang; pada implementasi ini mayoritas ditangani sebagai 401.
5. Kenapa JWT cocok untuk backend kamu?
    - JWT stateless, ringan untuk API, dan memudahkan scaling horizontal karena server tidak perlu menyimpan session login.
6. Apa risiko jika SECRET_KEY bocor?
    - Token bisa dipalsukan; mitigasinya rotasi secret, invalidasi token lama, dan update environment secara terkontrol.

7. Kenapa pakai env vars, bukan hardcode?
    - Agar aman, mudah ganti konfigurasi per environment, dan sesuai prinsip 12-Factor App.
8. Kenapa ada token expiry?
    - Untuk membatasi window serangan jika token dicuri dan memaksa re-authentication secara berkala.
9. Satu poin reliability paling penting di backend kamu?
    - Startup readiness: backend menunggu database siap (wait-for-db) dan punya endpoint healthcheck untuk observability.
10. Jika dipindah ke PaaS, apa yang paling dulu dicek?

- DATABASE_URL, SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES, ALLOWED_ORIGINS, lalu verifikasi health endpoint dan koneksi DB.